

Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»
Факультет Программной инженерии и Компьютерной техники

Отчёт по Модулю №1
по Системам Искусственного Интеллекта
Базы знаний и онтологии.
Система поддержки принятия решений
Вариант 16

Выполнили:
поменяй на своё имя
поменяй на своё имя
Преподаватель:
Жданов Андрей
Дмитриевич

Санкт-Петербург 2026г.

Содержание

Аннотация	3
Введение	3
Цели модуля	3
Почему БЗ и онтологии полезны	3
База знаний в Prolog (JP1, Часть 1)	3
Домен и словарь	3
Требования к БЗ и как они выполнены	3
Структура файла	4
Примеры запросов	4
Онтология в Protégé (JP1, Часть 2)	4
Перевод Prolog → OWL	4
Минимальный состав онтологии	5
Иерархия классов	5
Object Properties	5
Кардинальности и ограничения	5
Трассируемость Prolog ↔ OWL	6
DSS на основе БЗ (JP2)	6
Формат входной строки	6
Конвейер обработки	7
Диалог	7
Сценарии	7
Тестирование и отладка	7
Тест-кейсы Prolog	7
Тест-кейсы DSS	7
Оценка и интерпретация	8
Prolog vs OWL	8
Идеи развития	8
Заключение	8

Аннотация

В рамках модуля выполнены три лабораторные задачи: построена база знаний (БЗ) на языке Prolog, описывающая игровые сущности Civilization VI, та же модель переведена в онтологию OWL (Protege), и на её основе разработана система поддержки принятия решений (DSS), которая по строке фиксированного формата выдаёт игроку рекомендации. Домен - компьютерная игра Civilization VI (юниты, эпохи, технологии, ресурсы, цивилизации). DSS использует логический вывод Prolog: правила композиции, отрицания и кванторов позволяют отбирать юнитов, доступных игроку в текущей эре с учётом его ресурсов и стиля игры. Итог - механизм рекомендаций, где знания отделены от кода, а добавление нового юнита требует правки только .pl-файла.

Введение

Цели модуля

- освоить логическое моделирование предметной области
- научиться проектировать онтологии в OWL и проверять их ризонером
- понять, как строить системы поддержки принятия решений (DSS), опирающиеся на формальные знания, а не на жёстко прописанные правила.

Почему БЗ и онтологии полезны

- **Повторное использование знаний:** одна и та же модель используется и в Prolog, и в OWL, и в DSS.
- **Проверяемый вывод:** каждое решение можно объяснить - «юнит X доступен, потому что эра Y и ресурс Z».
- **Прозрачность:** правила видно, их можно читать и обсуждать, в отличие от весов нейросети.
- **Расширяемость:** чтобы добавить новую сущность, правим только базу знаний, а не код программы.

База знаний в Prolog (ЛР1, Часть 1)

Домен и словарь

Домен - Civilization VI. Сущности: **цивилизация, эпоха, юнит, технология, ресурс, район, здание**. Отношения: requires_resource/2, available_in_era/2, unlocks_unit/2, civ_unique_unit/2, building_in_district/2, era_before/2, unit_style/2.

Требования к БЗ и как они выполнены

Требование	Выполнено
≥ 20 фактов с 1 аргументом	78
10–15 фактов с 2 аргументами	100+
5–7 правил	9 правил: military_unit/1, unit_available_in_era/2, military_units_in_era/2, units_requiring_resource/2, unique_unit_of/2, buildings_in_district/2, unit_buildable/3, unit_unlocked_by_resource/4, best_unit_for_style/4

Структура файла

Файл `knowledge_base.pl` разбит на разделы:

- факты-классы (`unit_type/1`, `civilization/1`, `era/1`, ...)
- факты-свойства (`requires_resource/2`, `available_in_era/2`, ...)
- правила вывода (`unit_available_in_era/2`, `unit_buildable/3`, ...)
- комментарии-примеры тест-запросов.

Примеры запросов

 <code>civilization(X).</code> <code>X = russia</code> <code>X = egypt</code> <code>X = china</code> <code>X = greece</code> <code>X = persia</code> <code>X = mongolia</code> <code>X = rome</code> <code>X = scotland</code> <code>X = japan</code> <code>X = zulu</code> <code>X = korea</code>	 <code>military_units_in_era(classical, U).</code> <code>U = warrior</code> <code>U = archer</code> <code>U = horseman</code> <code>U = catapult</code> <code>U = swordsman</code> <code>U = scout</code>
 <code>requires_resource(U, iron), requires_resource(U, horses).</code> <code>U = knight</code> <code>false</code>	 <code>military_units_in_era(classical, U), \+ requires_resource(U, _).</code> <code>U = warrior</code> <code>U = archer</code> <code>U = horseman</code> <code>U = catapult</code> <code>U = scout</code>
 <code>civ_unique_unit(Civ, U).</code> <code>Civ = russia,</code> <code>U = cossack</code> <code>Civ = egypt,</code> <code>U = maryannu_chariot_archer</code> <code>Civ = china,</code> <code>U = chu_ko_nu</code> <code>Civ = greece,</code> <code>U = hoplite</code>	 <code>unit_buildable(U, classical, [iron]).</code> <code>U = warrior</code> <code>U = archer</code> <code>U = horseman</code> <code>U = catapult</code> <code>U = swordsman</code> <code>U = scout</code>

Онтология в Protégé (ЛР1, Часть 2)

Перевод Prolog → OWL

Онтология собрана в Protégé 5.6 на основе базы знаний из ЛР1. Соответствие между конструкциями Prolog и OWL:

Prolog	OWL	Комментарий
<code>unit_type(warrior).</code>	<code>civ:Warrior rdf:type civ:MilitaryUnit</code>	индивид класса
<code>military_unit(X) :- ...</code>	<code>civ:MilitaryUnit ⊆ civ:UnitType</code>	подкласс + disjoint
<code>requires_resource(U, R)</code>	<code>civ:requiresResource (ObjectProperty)</code>	domain: UnitType, range: Resource
<code>available_in_era(U, E)</code>	<code>civ:availableInEra (ObjectProperty)</code>	domain: UnitType, range: Era
<code>unlocks_unit(T, U)</code>	<code>civ:unlocksUnit (ObjectProperty)</code>	domain: Technology, range: UnitType

civ_unique_unit(C, U)	civ:civUniqueUnit (ObjectProperty)	domain: Civilization, range: UnitType
building_in_district(B, D)	civ:buildingInDistrict (ObjectProperty)	domain: Building, range: District
unit_style(U, melee)	civ:unitStyle (ObjectProperty)	domain: UnitType, range: Style
era_before(A, B)	civ:eraBefore (ObjectProperty, Transitive)	domain: Era, range: Era
name(X, 'Warrior')	civ:name (DataProperty, xsd:string)	строковое имя

Минимальный состав онтологии

Иерархия классов

```

owl:Thing
├── UnitType
│   ├── MilitaryUnit
│   │   └── CivilizationUnit
│   └── NonMilitaryUnit
├── Civilization
├── Era
├── Technology
├── Resource
├── Building
├── District
└── Style

```

Object Properties

Свойство	Domain	Range
requiresResource	UnitType	Resource
availableInEra	UnitType	Era
unlocksUnit	Technology	UnitType
civUniqueUnit	Civilization	UnitType
buildingInDistrict	Building	District
unitStyle	UnitType	Style
eraBefore	Era	Era

Свойство eraBefore объявлено транзитивным (owl:TransitiveProperty), что позволяет ризонеру выводить не прямые связи между эрами.

Кардинальности и ограничения

На ключевых отношениях заданы кардинальные ограничения:

- availableInEra exactly 1 Era — каждый юнит появляется ровно в одну эру;
- requiresResource max 2 Resource — юнит требует не более двух ресурсов.

Ограничение совместимости — owl:AllDisjointClasses между MilitaryUnit и NonMilitaryUnit: ризонер поймает противоречие, если один индивид попадёт в оба класса. Транзитивность eraBefore исключает циклы в порядке эр — при попытке замкнуть цепочку ризонер выведет eraBefore(X, X) и пометит онтологию как неконсистентную.

Трассируемость Prolog ↔ OWL

Prolog	OWL	Комментарий
unit_type(X)	civ:UnitType (класс)	все юниты — индивиды
military_unit(X) :- ...	civ:MilitaryUnit + disjoint	правило заменено явной типизацией
requires_resource(U, R)	civ:requiresResource	ObjectProperty
available_in_era(U, E)	civ:availableInEra	ObjectProperty
unlocks_unit(T, U)	civ:unlocksUnit	ObjectProperty
civ_unique_unit(C, U)	civ:civUniqueUnit	ObjectProperty
building_in_district(B, D)	civ:buildingInDistrict	ObjectProperty
unit_style(U, melee)	civ:unitStyle	Style — класс с индивидами melee/ranged
era_before(A, B)	civ:eraBefore (Transitive)	встроенная транзитивность
name(X, 'Warrior')	civ:name	DataProperty, xsd:string
unit_buildable/3, unit_unlocked_by_resource/4, best_unit_for_style/4	—	правила с forall и отрицанием не выражаются в OWL без SWRL; остаются в Prolog

DSS на основе БЗ (JLP2)

Формат входной строки

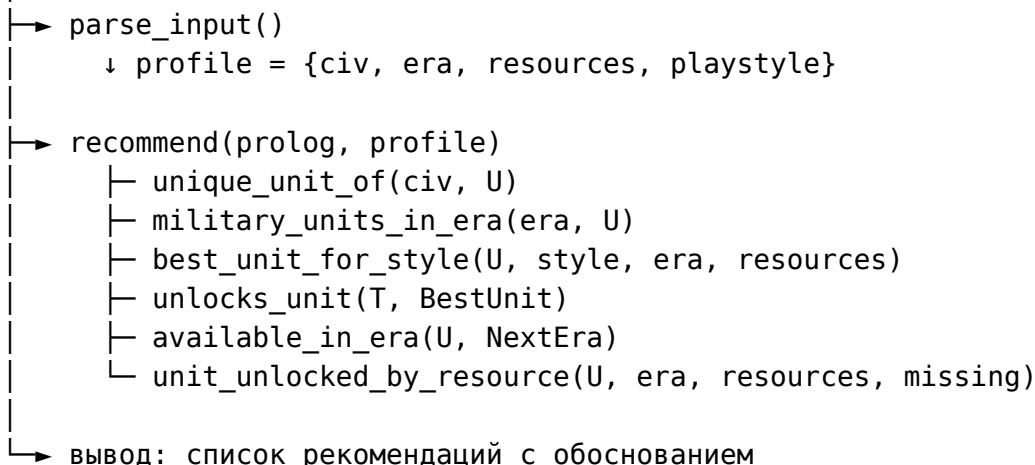
Я играю за <civ>[, сейчас эра <era>][, ресурсы <r1>[, <r2>]][, стиль melee|ranged]

Пример: Я играю за rome, сейчас эра classical, ресурсы iron, стиль melee

```
Шаблон: Я играю за <civ>[, сейчас эра <era>][, ресурсы <r1>, <r2>]][, стиль melee|ranged]
Цивилизации: russia, egypt, china, greece, persia, mongolia, rome, scotland, japan, zulu, korea
Эпохи: ancient, classical, medieval, renaissance, industrial, modern, information
Ресурсы: iron, horses
> Я играю за japan, сейчас эра medieval, ресурсы iron, стиль ranged
Рекомендации:
- Твой уникальный юнит: samurai
- В эту эру доступны юниты: warrior, archer, horseman, catapult, swordsman, crossbowman, trebuchet, scout
- Лучший юнит для твоего стиля игры: archer
- Чтобы строить archer, нужна технология: archery
- В следующей эре (renaissance) откроются: caravel
- Если добудешь horses, сможешь строить: knight
>
```

Конвейер обработки

строка



Диалог

Диалог с пользователем сводится к одной строке фиксированного формата. При неполных данных (например, не указана эра) DSS выдаёт общие рекомендации по всем военным юнитам с учётом ресурсов.

Сценарии

Вход	Выход
Я играю за rome	Уникальный юнит: legion. Все доступные юниты при отсутствии ресурсов
Я играю за rome, сейчас эра classical, ресурсы iron	Swordsman - лучший melee, чтобы строить, нужна iron_working
Я играю за japan, сейчас эра medieval, ресурсы iron, стиль ranged	Crossbowman - лучший ranged, технология military_tactics
Я играю за france	Ошибка: неизвестная цивилизация
asdkjhasd	Ошибка: строка не по шаблону

Тестирование и отладка

Тест-кейсы Prolog

Запрос	Ожидание
military_units_in_era(classical, U)	6 юнитов
unique_unit_of(japan, U)	samurai
best_unit_for_style(U, melee, classical, [iron])	swordsman
unit_unlocked_by_resource(U, classical, [iron], horses)	horseman

Тест-кейсы DSS

- Happy path: полная строка → 4–6 рекомендаций.
- Негативные: неизвестная цивилизация, неизвестная эра, неизвестный ресурс, строка без «Я играю за».

Оценка и интерпретация

Prolog vs OWL

Prolog	OWL/Protégé
Правила выводятся естественно (: -, \+, forall)	Правила требуют SWRL или ограничений, менее гибко
Быстро расширяется одним фактом	Расширение = правки в GUI, дольше
Хорош для DSS	Хорош для документации и валидации модели

Идеи развития

- добавить больше юнитов, зданий, технологий
- учитывать стоимость и силу юнитов
- интерактивный диалог с несколькими строками, а не одна команда
- подключить SPARQL-эндпоинт к OWL-версии.

Заключение

В ходе модуля построена база знаний Prolog, переведена в онтологию OWL, и на её основе реализована DSS-программа, которая по строке фиксированного формата выдаёт рекомендации. Механизм полностью основан на логическом выводе: добавление нового юнита требует только правки .pl-файла. Ограничения: нет стоимости и силы юнитов, диалог одношаговый, OWL-версия не подключена к DSS.